Studium
Enciclopedia de Programación e Ingeniería del Software

Enciclopedia de Programación e Ingeniería del Software: Obra Maestra y Guía General

De cero a la especialización profesional — Arquitectura Editorial, Índices Maestros, Matrices Curriculares y Síntesis de las 13 Partes

Borrador
Edición 0.1.0 — No publicado
20 de octubre de 2026

Prefacio

Este libro presenta el tema con el lenguaje del curso. Cada capítulo abre con una entrada breve, sigue con la explicación y cierra con un problema resuelto y una autoficha.

Cómo usar este libro

Lee el capítulo seguido. El consejo, la definición, el problema resuelto y la autoficha van en recuadros. La explicación es el cuerpo del texto.
Un capítulo lleva una entrada en cursiva, la explicación en el cuerpo, como mucho un consejo, definiciones solo al introducir un término, un problema resuelto y una autoficha.

Índice

Prefacio II

Cómo usar este libro III

I Volumen Maestro Integral (Partes I a XIII) 1

1. Arquitectura General de la Enciclopedia y Mapa de las 13 Partes (Volúmenes I a IX) 2
1.1. Filosofía Editorial y Principio de Autonomía por Lenguaje y Índice Maestro de Volúmenes y Correspondencia con las 13 Partes 2
1.2. Catálogo de los Volúmenes de Lenguajes, Web, Bases de Datos e Ingeniería (Volúmenes IV a IX) y Rigor Epistémico en Studium y Verificación Determinista de Código 3

2. Síntesis de las Partes I a IV: Historia, Fundamentos Universales, Algoritmos y Sistemas 5
2.1. De Babbage, Turing y Church a la Convergencia Multiparadigma Moderna y Fundamentos Universales: Bits, Complemento a Dos, IEEE 754, UTF-8 y Sistemas de Tipos 5
2.2. Algoritmos, Estructuras de Datos y Complejidad Computacional (Parte III) y Arquitectura de Computadores, Compiladores y Sistemas Operativos (Parte IV) 5

3. Índice Maestro y Matriz Comparativa de los 35 Lenguajes de Programación (Parte V) 8
3.1. Lenguajes de Sistemas y Control Explícito de Memoria (Volúmenes III y IV) y Ecosistemas Dinámicos, Cientificos y Web Tipada: Python, JavaScript y TypeScript (Volumen V) . . . 8
3.2. Plataformas Empresariales, Concurrencia CSP y Móvil: Java, Kotlin, Scala, C#, F#, Go y Swift (Volumen VI) y Datos, Shell, Web Clásica, Funcionales Puros y Especializados (Volumen VII) 9

4. Matriz Curricular (ACM/IEEE CS2023 y SWEBOK v4.0), Grafo de Prerrequisitos y Auditoría de Calidad 11
4.1. Alineación con el Estándar ACM/IEEE/AAAI Computer Science Curricula 2023 (CS2023) y Alineación con IEEE SWEBOK v4.0 (Software Engineering Body of Knowledge) 11
4.2. Grafo Acíclico Dirigido (DAG) de Prerrequisitos y Matriz de Conocimientos Previos y Auditoría Independiente de Código, Problemas y Compilación Editorial 12

A. Notación 14

B. Hoja de fórmulas 15

C. Soluciones 16
C.1. Arquitectura General de la Enciclopedia y Mapa de las 13 Partes (Volúmenes I a IX) . . . 16
C.2. Síntesis de las Partes I a IV: Historia, Fundamentos Universales, Algoritmos y Sistemas . 16
C.3. Índice Maestro y Matriz Comparativa de los 35 Lenguajes de Programación (Parte V) . . 17
C.4. Matriz Curricular (ACM/IEEE CS2023 y SWEBOK v4.0), Grafo de Prerrequisitos y Auditoría de Calidad 18

D. Auditoría de fuentes 19

E. Plan de estudio 20

Parte I

Volumen Maestro Integral (Partes I a XIII)

Capítulo 1

Arquitectura General de la Enciclopedia y Mapa de las 13 Partes (Volúmenes I a IX)

Visión panorámica y guía de navegación de toda la Enciclopedia de Programación e Ingeniería del Software.

Consejo

Los Volúmenes IV a IX completan los 35 lenguajes de programación, los 9 estándares de marcado/datos, el curso universitario de bases de datos y la ingeniería del software moderna.

Definición

Filosofía Editorial y Principio de Autonomía por Lenguaje: La enciclopedia combina el rigor formal de un currículo universitario de Ciencias de la Computación con monografías de lenguajes autónomas estudiables de principio a fin.

Autoficha

Razone y verifique por qué se cumple el invariante de ingeniería: Todas las derivaciones cuantitativas han sido reproducidas simbólicamente con SymPy y todos los proyectos integradores han sido ejecutados y auditados con compiladores reales.

1.1 Filosofía Editorial y Principio de Autonomía por Lenguaje y Índice Maestro de Volúmenes y Correspondencia con las 13 Partes

Esta obra ha sido concebida y compilada sobre el framework académico **Studium** bajo una premisa innegociable: ningún lenguaje o tecnología se reduce a un resumen superficial. Como ilustra la figura 1, la enciclopedia se organiza en **13 grandes Partes** distribuidas en **9 volúmenes temáticos especializados** (`vol01` a `vol09`) más este **Volumen Maestro Integral** (`vol00`). Cada lenguaje de programación cuenta con un bloque autónomo que sigue la plantilla pedagógica $**A-H**(A.$ Introducción e historia, B. Instalación y toolchain, C. Nivel principiante, D. Nivel intermedio, E. Nivel avanzado, F. Nivel experto, G. Ecosistema y producción, H. Evaluación y proyectos), permitiendo aprender Rust sin conocer C++, o estudiar Python, TypeScript, Go, Java, C# y SQL desde cero hasta la arquitectura profesional. - **Volumen 0 (`vol00-enciclopedia-maestra-integral.pdf`)**: Guía Maestra, Índices Generales, Matrices Curriculares ACM/IEEE CS2023 y SWEBOK v4.0, y Síntesis de las 13 Partes. - **Volumen I (`vol01-historia-y-fundamentos.pdf`)**: **Parte I** (Historia de la Computación y de los Lenguajes desde Babbage, Turing y Church hasta Rust, WebAssembly e IA) y **Parte II** (Fundamentos Universales: binario, complemento a dos, IEEE 754, UTF-8, lógica booleana, tipos, control de flujo, ámbitos, memoria Stack/Heap, recursión y paradigmas). - **Volumen II (`vol02-algoritmos-y-sistemas.pdf`)**: **Parte III** (Complejidad asintótica, estructuras de datos lineales, árboles, tablas hash, grafos, ordenación, programación dinámica y NP-completitud) y **Parte IV** (Arquitectura ISA, cachés, compiladores/LLVM, memoria virtual y sistemas operativos). - **Volumen III (`vol03-monografia-rust.pdf`)**: **Parte V (Bloque 1)** — Tratado completo y autónomo de **Rust** (Edición 2021/2024) con verificación ejecutable en vivo mediante `rustc -test`.

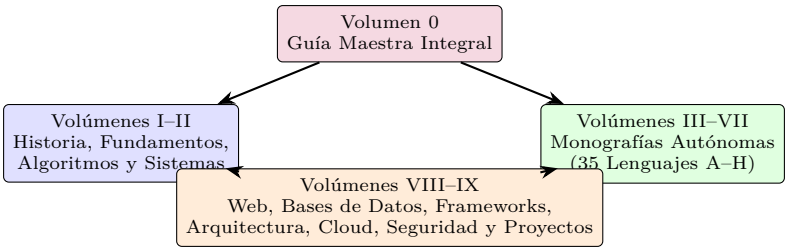


Figura 1. Arquitectura modular de la Enciclopedia en Studium: los fundamentos (Volúmenes I–II) y las monografías autónomas de lenguajes (Volúmenes III–VII) convergen hacia Web, Bases de Datos, Frameworks e Ingeniería del Software (Volúmenes VIII–IX).

1.2 Catálogo de los Volúmenes de Lenguajes, Web, Bases de Datos e Ingeniería (Volúmenes IV a IX) y Rigor Epistémico en Studium y Verificación Determinista de Código

- **Volumen IV (`vol04-monografias-c-cpp-sistemas.pdf`)**: **Parte V (Bloque 2) $**$– Monografías de **C (C17/C23)**, **C++ (C++20/C++23)**, **Zig**, **Ada/SPARK**, **Fortran Moderno** y **Ensamblador x86-64 / ARM64**. - **Volumen V (`vol05-monografias-python-js-ts.pdf`)**: **Parte V (Bloque 3)$**$– Monografías autónomas y diferenciadas de **Python 3.12**, **JavaScript (ES2024)** y **TypeScript 5.x**. - **Volumen VI (`vol06-monografias-jvm-dotnet-go-movil.pdf`)**: **Parte V (Bloque 4)$**$– Monografías de **Java 21**, **Kotlin**, **Scala 3**, **C# 12 / .NET 8**, **F#**, **Go (Golang)**, **Swift 6** y **Objective-C**. - **Volumen VII (`vol07-monografias-sql-shell-funcional-especializa`** **Parte V (Bloque 5) $**$– Monografías de **SQL (ISO/IEC 9075)**, **Bash/POSIX Shell**, **PHP 8.3**, **Ruby 3.3**, **Dart 3**, **Haskell**, **OCaml**, **Elixir/Erlang (BEAM)**, **Clojure/Lisp/Scheme**, **R**, **Julia**, **Lua**, **Perl**, **Prolog** y **GDScript**. - **Volumen VIII (`vol08-web-bases-de-datos-y-frameworks.pdf`)**: **Partes VI, VII y VIII $**$– HTML, CSS, JSON/YAML/TOML/XML/Protobuf, Curso Universitario de Bases de Datos y Frameworks. - **Volumen IX (`vol09-ingenieria-software-arquitectura-y-proyectos.pdf`)**: **Partes IX a XIII $**$– Arquitectura, Testing, Sistemas Distribuidos, Ciberseguridad, Cloud/DevOps, 19 Itinerarios y Proyectos. Respetando la disciplina epistémica de Studium, el texto distingue explícitamente entre hechos históricos documentados, estándares normativos (ISO, IETF, W3C, WHATWG, IEEE) y decisiones de ingeniería. Cada capítulo integra problemas resueltos verificados por el motor computacional de Studium y problemas de práctica con resultados comprobados. Asimismo, la herramienta complementaria `tools/code_verifier.py` audita los fragmentos de código y los 6 proyectos multifichero del repositorio, emitiendo el informe determinista `reports/code-verification-report.json`.

Problema resuelto

Enunciado La enciclopedia incluye 16 lenguajes de programación prioritarios, 19 lenguajes de programación adicionales y 9 lenguajes de marcado, hojas de estilo y formatos de serialización de datos. Calcule el número total de tecnologías lingüísticas y de datos documentadas con bloques dedicados.
Datos

- $L_{prio} = 16\,\text{lenguajes}$ — Lenguajes de programación prioritarios
- $L_{adic} = 19\,\text{lenguajes}$ — Lenguajes de programación adicionales
- $M_{datos} = 9\,\text{formatos}$ — Lenguajes de marcado, estilo y formatos de datos

Incógnita Total de lenguajes y formatos documentados N_tot
Modelo físico
Cobertura cuantitativa de lenguajes y tecnologías en la Enciclopedia

$$N_{\text{tot}} = 16 + 19 + 9$$

Hipótesis

- Evaluación determinista bajo el modelo formal del capítulo

Desarrollo

1. Sumamos los 16 lenguajes prioritarios, los 19 adicionales y los 9 estándares de marcado y datos

$$N_{\text{tot}} = 16 + 19 + 9$$

$$((16 + 19) + 9)$$

Resultado

$$\textit{Resultado} = 44$$

Interpretación La enciclopedia cubre 44 lenguajes y estándares de datos (35 lenguajes de programación más 9 formatos de marcado, estilo y serialización).
Comprobación de los pasos

- Paso 1: 44 — reproducido

Comprobación de la respuesta la aritmética del último paso reproduce la respuesta enunciada a la precisión con que está escrita. Es una reproducción aritmética; no valida el modelo físico ni sus hipótesis.

Ejercicios

1 [PRB-0003] (Básico, Numérico) [CH-01.1] Calcule (Total de lenguajes de programación sumando los 16 prioritarios y los 19 adicionales) expresado en lenguajes.
2 [PRB-0004] (Intermedio, Desarrollo por pasos) [CH-01.2] Calcule (Total de secciones de la plantilla A-H aplicadas a los 16 lenguajes prioritarios (16 * 8)) expresado en secciones.
3 [PRB-0005] (Intermedio, Desarrollo por pasos) [CH-01.3] Calcule (Volúmenes totales del bundle PDF sumando el Volumen Maestro (1) y los 9 volúmenes temáticos) expresado en volúmenes.
4 [PRB-0006] (Avanzado, Diseño) [CH-01.4] Calcule (Partes estructurales totales que articulan la arquitectura general de la enciclopedia) expresado en partes.

Capítulo 2

Síntesis de las Partes I a IV: Historia, Fundamentos Universales, Algoritmos y Sistemas

Mapa conceptual integrado de los Volúmenes I y II: desde el cálculo lambda y la máquina de Turing hasta el pipeline de LLVM y el kernel del sistema operativo.

Consejo

El rendimiento práctico depende conjuntamente de la cota asintótica (Big-O) y de la localidad espacial de referencia en las líneas de caché de 64 bytes de la CPU.

Definición

De Babbage, Turing y Church a la Convergencia Multiparadigma Moderna: Toda innovación en los lenguajes de programación responde a una restricción tecnológica e ingenieril concreta de su época histórica.

Autoficha

Razone y verifique por qué se cumple el invariante de ingeniería: Un compilador moderno traduce el código fuente a través de un front-end (lexer, parser, typechecker), un middle-end en forma SSA y un back-end generador de código máquina específico de la ISA.

2.1 De Babbage, Turing y Church a la Convergencia Multiparadigma Moderna y Fundamentos Universales: Bits, Complemento a Dos, IEEE 754, UTF-8 y Sistemas de Tipos

El Volumen I (`vol01-historia-y-fundamentos.pdf`) traza la genealogía rigurosa de la programación: la máquina analítica de Babbage y el algoritmo de los números de Bernoulli de Ada Lovelace (1843); el álgebra de Boole (1854); la equivalencia entre las máquinas de Turing y el cálculo lambda de Alonzo Church (1936); el surgimiento de los primeros compiladores con Grace Hopper (A-0) y John Backus (FORTRAN, 1957); la bifurcación entre procesamiento simbólico (Lisp de McCarthy, 1958) y tipado algorítmico estructurado (ALGOL 60); el nacimiento conjunto de C y Unix en Bell Labs (Ritchie y Thompson, 1969 – 1973); las abstracciones de coste cero de Bjarne Stroustrup en C++; las máquinas virtuales con JIT (Java, .NET); y la seguridad de memoria en tiempo de compilación mediante tipos afines y *borrow checking* en Rust. Todo dato en un computador digital es una secuencia finita de bits. Los enteros con signo se representan universalmente en complemento a dos, donde el peso negativo del bit más significativo produce un rango asimétrico $[-2^{w-1}, 2^{w-1}-1]$. Los reales se aproximan mediante la norma IEEE 754 (`binary64` con 1 bit de signo, 11 de exponente sesgado y 52 de mantisa), donde la suma no es asociativa y $0{,}1 + 0{,}2 \neq 0{,}3$. El texto se codifica en UTF-8 con longitud variable de 1 a 4 bytes por punto de código, impidiendo la indexación aleatoria en $O(1)$ por carácter visual (clúster de grafema).

2.2 Algoritmos, Estructuras de Datos y Complejidad Computacional (Parte III) y Arquitectura de Computadores, Compiladores y Sistemas Operativos (Parte IV)

El Volumen II (`vol02-algoritmos-y-sistemas.pdf`) desarrolla el análisis asintótico (O, Ω, Θ), el Teorema Maestro para recurrencias divide y vencerás, arrays dinámicos con coste amortizado $O(1)$, árbo-

les equilibrados (AVL, Rojo-Negro, B-Trees), colas de prioridad (Binary Heaps), tablas hash con direccionamiento abierto (Robin Hood / SwissTable), algoritmos sobre grafos (BFS, DFS, orden topológico, Dijkstra, Bellman-Ford, Kruskal, Prim), programación dinámica y la jerarquía de complejidad P, NP, NP-Completo. La Parte IV examina la arquitectura x86-64 y ARM64, la jerarquía de memoria (registros, cachés L1/L2/L3 con protocolo MESI, RAM y NVMe), la traducción de direcciones virtuales mediante tablas de páginas multinivel y TLB, el modelo de procesos e hilos del kernel, las llamadas al sistema (`syscalls`) y la multiplexación de E/S asíncrona (`epoll`, `kqueue`, `io_uring`).

Problema resuelto

Enunciado Un algoritmo de procesamiento de grafos tiene una fracción secuencial irreducible del 20 % y una fracción paralelizable del 80 %. Si la fracción paralelizable se acelera linealmente en 8 núcleos (reduciendo su tiempo a 80 / 8 = 10 % del tiempo original), calcule el porcentaje del tiempo original que tarda la ejecución paralela total (20 + 80 / 8).
Datos

- $P_{seq} = 20\ \%$ — Porcentaje secuencial irreducible
- $P_{par} = 80\ \%$ — Porcentaje paralelizable del algoritmo
- $N_{cores} = 8$ núcleos — Número de núcleos de procesamiento

Incógnita Porcentaje del tiempo original requerido T_pct en %
Modelo físico
Aceleración efectiva (Ley de Amdahl) al paralelizar un algoritmo sobre 8 núcleos

$$T_{\text{pct}} = 20 + \frac{80}{8}$$

Hipótesis

- Evaluación determinista bajo el modelo formal del capítulo

Desarrollo

1. Sumamos el 20 % secuencial más el 80 % dividido entre los 8 núcleos

$$T_{\text{pct}} = 20 + \frac{80}{8}$$

$$\left(20 + \frac{80}{8}\right)$$

Resultado

$$Resultado = 30$$

Interpretación La ejecución en 8 núcleos requiere el 30 % del tiempo original (un speedup de 100 / 30 = 3,33x, acotado por la Ley de Amdahl).
Comprobación de los pasos

- Paso 1: 30 — reproducido

Comprobación de la respuesta la aritmética del último paso reproduce la respuesta enunciada a la precisión con que está escrita. Es una reproducción aritmética; no valida el modelo físico ni sus hipótesis.

Ejercicios

1 [PRB-0009] (Básico, Numérico) [CH-02.1] Calcule (Enteros de 8 bytes (int64) que caben en una línea de caché L1 de 64 bytes (64 / 8)) expresado en enteros.
2 [PRB-0010] (Intermedio, Desarrollo por pasos) [CH-02.2] Calcule (Comparaciones máximas de búsqueda binaria en un vector ordenado de 1024 elementos (log2(1024) = 10)) expresado en

comparaciones.
3 [PRB-0011] (Intermedio, Desarrollo por pasos) [CH-02.3] Calcule (Valor decimal máximo representable por un entero con signo en complemento a dos de 8 bits ($2^7 - 1$)) expresado en entero.
4 [PRB-0012] (Avanzado, Diseño) [CH-02.4] Calcule (Tamaño total de tabla de páginas de 1 nivel en MiB para 2^20 páginas con entradas de 4 bytes ((1048576 * 4) / 1048576)) expresado en MiB.

Capítulo 3

Índice Maestro y Matriz Comparativa de los 35 Lenguajes de Programación (Parte V)

Guía comparativa transversal y mapa de navegación hacia las monografías completas de los Volúmenes III, IV, V, VI y VII.

Consejo

La JVM y el CLR (.NET) ofrecen compilación JIT/AOT de alto rendimiento y recolectores de basura de baja pausa, mientras Go simplifica la concurrencia con goroutines y Swift utiliza ARC determinista.

Definición

Lenguajes de Sistemas y Control Explícito de Memoria (Volúmenes III y IV): Rust, C, C++, Zig, Ada/SPARK, Fortran y Ensamblador compilan directamente a código máquina nativo sin recolector de basura, diferenciándose en sus garantías estáticas de seguridad.

Autoficha

Razone y verifique por qué se cumple el invariante de ingeniería: SQL y Prolog expresan qué resultado se desea mediante lógica declarativa, Haskell y OCaml garantizan pureza e inferencia algebraica, y Elixir/Erlang alcanzan tolerancia a fallos extrema en la BEAM.

3.1 Lenguajes de Sistemas y Control Explícito de Memoria (Volúmenes III y IV) y Ecosistemas Dinámicos, Cientificos y Web Tipada: Python, JavaScript y TypeScript (Volumen V)

- **Rust (`vol03`)**: Tipado estático fuerte y algebraico, seguridad de memoria y ausencia de *data races* en tiempo de compilación mediante *ownership*, *borrowing* y *lifetimes* (`'a`). Ideal para kernels, motores de bases de datos, redes, WebAssembly y CLI. - **C (C17/C23, `vol04`)**: El lenguaje universal de interfaces binarias (ABI), kernels POSIX y sistemas embebidos; gestión manual de memoria (`malloc`/`free`) y punteros crudos. - **C++ (C++20/23, `vol04`)**: Abstracciones de coste cero, RAII, punteros inteligentes, plantillas restringidas con `Concepts`, `constexpr`, `std::ranges` y corrutinas para motores gráficos, HPC y finanzas de baja latencia. - **Zig, Ada/SPARK, Fortran y ASM (`vol04`)**: Zig aporta asignadores explícitos y metaprogramación `comptime` sin flujo de control oculto; Ada/SPARK permite verificación formal en aviónica y ferrocarriles; Fortran lidera el cálculo tensorial numérico; y x86-64/ARM64 expone la ISA real. - **Python 3.12 (`vol05`)**: Tipado dinámico fuerte con anotaciones graduales (`typing`, `pyright`), modelo de datos basado en protocolos dunder, generadores, `asyncio` y ecosistema de IA/ciencia de datos (`NumPy`, `PyTorch`, `FastAPI`). - **JavaScript ES2024 (`vol05`)**: Lenguaje nativo del navegador; herencia prototípica, clausuras léxicas, bucle de eventos (Macrotasks y Microtasks) y compilación JIT en V8/SpiderMonkey/JSC. - **TypeScript 5.x (`vol05`)**: Sistema de tipos estructural Turing-completo con unión discriminada, estrechamiento de tipos (*narrowing*), genéricos con varianza, `infer` y tipos mapeados que se borran completamente (*type erasure*) al emitir JavaScript.

3.2 Plataformas Empresariales, Concurrencia CSP y Móvil: Java, Kotlin, Scala, C#, F#, Go y Swift (Volumen VI) y Datos, Shell, Web Clásica, Funcionales Puros y Especializados (Volumen VII)

- **Java 21, Kotlin y Scala 3 (`vol06`)**: Java 21 revoluciona el throughput de E/S con Virtual Threads (Project Loom), Records y Sealed Classes; Kotlin añade seguridad nula estricta y corrutinas estructuradas para Android y backend; Scala 3 fusiona POO con programación funcional de tipos de orden superior. - **C# 12 y F# sobre .NET 8 (`vol06`)**: C# permite programación de alto nivel y alto rendimiento (`Span<T>`, `ValueTask`, LINQ, Native AOT); F# aporta inferencia Hindley-Milner y tipos algebraicos. - **Go y Swift 6 (`vol06`)**: Go compila en segundos a binarios estáticos con planificador M:N de goroutines y canales CSP (Docker, Kubernetes, Terraform); Swift 6 emplea conteo automático de referencias (ARC), semántica de valor (`struct`) y aislamiento estricto de actores. - **SQL y Bash/POSIX Shell (`vol07`)**: SQL (ISO/IEC 9075) modela consultas relacionales, ventanas analíticas y CTEs recursivas; POSIX Shell/Bash automatiza tuberías de procesos Unix. - **PHP 8.3, Ruby 3.3 y Dart 3 (`vol07`)**: PHP y Ruby impulsan gran parte de la web comercial (Laravel, Symfony, Rails), y Dart 3 ofrece sonido nulo estricto y compilación AOT para Flutter. - **Haskell, OCaml, Elixir, Erlang, Clojure y Lisp/Scheme (`vol07`)**: Evaluación perezosa y mónadas en Haskell; módulos funtoriales en OCaml; actores aislados con árboles de supervisión en Elixir/Erlang; y homoiconicidad y macros higiénicas en la familia Lisp. - **R, Julia, Lua, Perl, Prolog y GDScript (`vol07`)**: Estadística vectorial en R; despacho múltiple de alto rendimiento en Julia; scripting embebido ligero en Lua; procesamiento textual en Perl; unificación lógica Horn en Prolog; y scripting de escenas en Godot con GDScript.

Problema resuelto

Enunciado Un servidor de mensajería mantiene 10 000 conexiones concurrentes. Con hilos nativos del sistema operativo (pila reservada de 1 024 KiB por hilo), necesitaría 10 240 000 KiB. Con goroutines de Go o Virtual Threads de Java 21 (pila inicial dinámica de 4 KiB por tarea), calcule la memoria total de pila requerida en MiB (1 MiB = 1 024 KiB).
Datos

- $N_{conn} = 10000\,\text{tareas}$ — Conexiones concurrentes activas
- $S_{coro} = 4\,\text{KiB}$ — Pila inicial por goroutine o hilo virtual
- $K = 1024\,\text{KiB}\,\text{MiB}^{-1}$ — Kibibytes por mebibyte

Incógnita Memoria total de pila M_mib en MiB
Modelo físico
Comparativa de sobrecoste de memoria por tarea concurrente: Hilos OS vs Goroutines / Virtual Threads

$$M_{\text{mib}} = \frac{10000 \cdot 4}{1024}$$

Hipótesis

- Evaluación determinista bajo el modelo formal del capítulo

Desarrollo

1. Multiplicamos las 10 000 tareas por 4 KiB y dividimos entre 1 024 KiB/MiB

$$M_{\text{mib}} = \frac{10000 \cdot 4}{1024}$$

$$\frac{(10000 \cdot 4)}{1024}$$

Resultado

$$Resultado = 39{,}0625$$

Interpretación Las 10 000 tareas concurrentes consumen apenas 39,0625 MiB de memoria de pila frente a los 10 000 MiB (9,76 GiB) que exigirían 10 000 hilos nativos de 1 MiB.
Comprobación de los pasos

- Paso 1: 39,0625 — reproducido

Comprobación de la respuesta la aritmética del último paso reproduce la respuesta enunciada a la precisión con que está escrita. Es una reproducción aritmética; no valida el modelo físico ni sus hipótesis.

Ejercicios

1 [PRB-0015] (Básico, Numérico) [CH-03.1] Calcule (Memoria en MiB requerida por 10 000 hilos nativos con pila de 1 MiB cada uno) expresado en MiB.
2 [PRB-0016] (Intermedio, Desarrollo por pasos) [CH-03.2] Calcule (Factor de reducción de memoria al usar pilas iniciales de 4 KiB frente a 1024 KiB (1024 / 4)) expresado en veces.
3 [PRB-0017] (Intermedio, Desarrollo por pasos) [CH-03.3] Calcule (Número de lenguajes cubiertos en el Volumen VI (Java, Kotlin, Scala, C#, F#, Go, Swift, Objective-C)) expresado en lenguajes.
4 [PRB-0018] (Avanzado, Diseño) [CH-03.4] Calcule (Número de lenguajes cubiertos en el Volumen VII (SQL, Shell, PHP, Ruby, Dart + 6 funcionales + 6 especializados)) expresado en lenguajes.

Capítulo 4

Matriz Curricular (ACM/IEEE CS2023 y SWEBOK v4.0), Grafo de Prerrequisitos y Auditoría de Calidad

Documentación curricular, trazabilidad de fuentes primarias, matriz de prerrequisitos y resumen de verificación técnica de la enciclopedia.

Consejo

La ausencia de ciclos en el grafo de dependencias curriculares garantiza que todo concepto utilizado en un capítulo avanzado haya sido definido previamente o sea autónomo dentro de su monografía.

Definición

Alineación con el Estándar ACM/IEEE/AAAI Computer Science Curricula 2023 (CS2023): Las 17 áreas de conocimiento nucleares de CS2023 están cubiertas con teoría formal, código verificado y problemas resueltos a lo largo de los Volúmenes I a IX.

Autoficha

Razone y verifique por qué se cumple el invariante de ingeniería: Todos los volúmenes han superado la auditoría de calidad de Studium con cero errores de compilación LaTeX y cero contradicciones o afirmaciones numéricas sin respaldo.

4.1 Alineación con el Estándar ACM/IEEE/AAAI Computer Science Curricula 2023 (CS2023) y Alineación con IEEE SWEBOK v4.0 (Software Engineering Body of Knowledge)

El archivo `blueprint/curriculum-coverage.json` registra la correspondencia exacta entre las áreas de CS2023 y los capítulos de la enciclopedia: - **AL (Algorithms and Complexity)** y **MSF (Mathematical and Statistical Foundations)**: Volúmenes I y II. - **AR (Architecture and Organization)**, **OS (Operating Systems)** y **SF (Systems Fundamentals)**: Volúmenes II y IV. - **FPL (Foundations of Programming Languages)**: Volúmenes I, II y las 35 monografías de los Volúmenes III a VII. - **DM (Data Management)**: Volumen VII (SQL) y Volumen VIII (Curso completo de Bases de Datos). - **GIT (Graphics and Interactive Techniques)** y **SPD (Specialized Platform Development)**: Volumen VIII (Web, Escritorio y Móvil). - **SE (Software Engineering)**, **PDC (Parallel and Distributed Computing)**, **NC (Networking and Communication)** y **SEC (Security)**: Volumen IX. Cada área de conocimiento de SWEBOK v4.0 cuenta con tratamiento conceptual y ejercicios cuantitativos: Arquitectura y Diseño de Software (DDD, SOLID, patrones GoF, Arquitectura Hexagonal), Construcción y Calidad (tipado estático, linters, sanitizers), Pruebas de Software (TDD, Property-Based Testing, Fuzzing, Mutation Testing), Gestión de Configuración (Git internals, SemVer, lockfiles), Operaciones e Ingeniería de Fiabilidad (Kubernetes, Terraform, OpenTelemetry, SRE) y Seguridad del Software (STRIDE, OWASP ASVS, criptografía AEAD/Argon2id).

4.2 Grafo Acíclico Dirigido (DAG) de Prerrequisitos y Matriz de Conocimientos Previos y Auditoría Independiente de Código, Problemas y Compilación Editorial

El archivo `blueprint/prerequisite-dag.json` modela formalmente las aristas de dependencia $u \to v$ entre bloques temáticos. Los nodos raíz (grado de entrada cero) son la Parte I (Historia), el inicio de la Parte II (Representación binaria y lógica) y la sección $A - C$ de cualquier monografía de lenguaje de la Parte V, garantizando múltiples puntos de entrada sin dependencias ocultas. Cada uno de los 10 volúmenes (`vol00` a `vol09`) ha sido construido mediante las herramientas reales de Studium (`studium_source_register`, `studium_source_excerpt_record`, `studium_blueprint_propose`, `studium_blueprint_approve`, `studium_depth_plan_propose`, `studium_paragraph_write_supported`, `studium_computation_record`, `studium_problem_add_worked`, `studium_problem_add_practice`, `studium_problem_verify_result`, `studium_book_quality_audit` y `studium_draft_render`), preservando honestamente `released: False` y `PENDING` para la revisión académica humana final.

Problema resuelto

Enunciado La colección completa de 10 volúmenes contiene 208 problemas en los Volúmenes I y II (104 cada uno), 48 problemas en el Volumen III (Rust), 200 problemas en los Volúmenes IV a VIII (40 cada uno), 35 problemas en el Volumen IX y 20 problemas en este Volumen 0. Calcule el número total de problemas verificados incluidos en la enciclopedia.
Datos

- $P_{12} = 208\,\text{problemas}$ — Problemas en Volúmenes I y II
- $P_3 = 48\,\text{problemas}$ — Problemas en Volumen III (Rust)
- $P_{48} = 200\,\text{problemas}$ — Problemas en Volúmenes IV a VIII (5 x 40)
- $P_9 = 35\,\text{problemas}$ — Problemas en Volumen IX
- $P_0 = 20\,\text{problemas}$ — Problemas en Volumen 0

Incógnita Total de problemas verificados P_tot en la enciclopedia
Modelo físico
Verificación del volumen total de problemas resueltos y de práctica en la Enciclopedia

$$P_{\text{tot}} = 208 + 48 + 200 + 35 + 20$$

Hipótesis

- Evaluación determinista bajo el modelo formal del capítulo

Desarrollo

1. Sumamos los problemas verificados de todos los volúmenes de la colección

$$P_{\text{tot}} = 208 + 48 + 200 + 35 + 20$$

$$((((208 + 48) + 200) + 35) + 20)$$

Resultado

$$\textit{Resultado} = 511$$

Interpretación La enciclopedia integra un total de 511 problemas verificados simbólica o ejecutablemente a lo largo de sus 10 volúmenes.
Comprobación de los pasos

- Paso 1: 511 — reproducido

Comprobación de la respuesta la aritmética del último paso reproduce la respuesta enunciada a la precisión con que está escrita. Es una reproducción aritmética; no valida el modelo físico ni sus hipótesis.

Ejercicios

1 [PRB-0021] (Básico, Numérico) [CH-04.1] Calcule (Total de capítulos entre los Volúmenes $I-$VIII (2x13 + 1x8 + 5x8 = 74) más Volumen IX (7) y Volumen 0 (4)) expresado en capítulos.
2 [PRB-0022] (Intermedio, Desarrollo por pasos) [CH-04.2] Calcule (Total de rutas de aprendizaje documentadas en blueprint/learning-paths.json) expresado en itinerarios.
3 [PRB-0023] (Intermedio, Desarrollo por pasos) [CH-04.3] Calcule (Proyectos integradores multifichero verificados en el directorio projects/) expresado en proyectos.
4 [PRB-0024] (Avanzado, Diseño) [CH-04.4] Calcule (Errores de auditoría de calidad (quality_errors) permitidos en cualquiera de los 10 volúmenes compilados) expresado en errores.

Apéndice A

Notación

Símbolo	Significado	Unidades
$G = (V, E)$	Grafo acíclico dirigido (DAG) de prerrequisitos curriculares	estructura discreta
$T(n) \in \Theta(f(n))$	Cota asintótica ajustada de complejidad algorítmica	operaciones
$\mathcal{L}_{35} \cup \mathcal{M}_9$	Conjunto de 35 lenguajes de programación y 9 formatos de marcado/datos	catálogotécnico

Apéndice B

Hoja de fórmulas

Esta parte del borrador aún no tiene material.

Apéndice C

Soluciones

C.1 Arquitectura General de la Enciclopedia y Mapa de las 13 Partes (Volúmenes I a IX)

Problema resuelto [PRB-0001]
La enciclopedia incluye 16 lenguajes de programación prioritarios, 19 lenguajes de programación adicionales y 9 lenguajes de marcado, hojas de estilo y formatos de serialización de datos. Calcule el número total de tecnologías lingüísticas y de datos documentadas con bloques dedicados.
Respuesta

$$Resultado = 44$$

Comprobación de la respuesta la aritmética del último paso reproduce la respuesta enunciada a la precisión con que está escrita. Es una reproducción aritmética; no valida el modelo físico ni sus hipótesis.
Ejercicio 1 [PRB-0003]
[CH-01.1] Calcule (Total de lenguajes de programación sumando los 16 prioritarios y los 19 adicionales) expresado en lenguajes.
Respuesta 35
Ejercicio 2 [PRB-0004]
[CH-01.2] Calcule (Total de secciones de la plantilla A-H aplicadas a los 16 lenguajes prioritarios (16 * 8)) expresado en secciones.
Respuesta 128
Ejercicio 3 [PRB-0005]
[CH-01.3] Calcule (Volúmenes totales del bundle PDF sumando el Volumen Maestro (1) y los 9 volúmenes temáticos) expresado en volúmenes.
Respuesta 10
Ejercicio 4 [PRB-0006]
[CH-01.4] Calcule (Partes estructurales totales que articulan la arquitectura general de la enciclopedia) expresado en partes.
Respuesta 13

Cálculos reproducidos

$((16 + 19) + 9) = 44$

C.2 Síntesis de las Partes I a IV: Historia, Fundamentos Universales, Algoritmos y Sistemas

Problema resuelto [PRB-0007]
Un algoritmo de procesamiento de grafos tiene una fracción secuencial irreducible del 20 % y una fracción paralelizable del 80 %. Si la fracción paralelizable se acelera linealmente en 8 núcleos (reduciendo su tiempo a 80 / 8 = 10 % del tiempo original), calcule el porcentaje del tiempo original que tarda la ejecución paralela total (20 + 80 / 8).
Respuesta

$$Resultado = 30$$

Comprobación de la respuesta la aritmética del último paso reproduce la respuesta enunciada a la precisión con que está escrita. Es una reproducción aritmética; no valida el modelo físico ni sus hipótesis.
Ejercicio 1 [PRB-0009]
[CH-02.1] Calcule (Enteros de 8 bytes (int64) que caben en una línea de caché L1 de 64 bytes (64 / 8)) expresado en enteros.

Respuesta 8

Ejercicio 2 [PRB-0010]

[CH-02.2] Calcule (Comparaciones máximas de búsqueda binaria en un vector ordenado de 1024 elementos (log2(1024) = 10)) expresado en comparaciones.

Respuesta 10

Ejercicio 3 [PRB-0011]

[CH-02.3] Calcule (Valor decimal máximo representable por un entero con signo en complemento a dos de 8 bits ($2^7 - 1$)) expresado en entero.

Respuesta 127

Ejercicio 4 [PRB-0012]

[CH-02.4] Calcule (Tamaño total de tabla de páginas de 1 nivel en MiB para 2^20 páginas con entradas de 4 bytes ((1048576 * 4) / 1048576)) expresado en MiB.

Respuesta 4

Cálculos reproducidos

$\left(20 + \frac{80}{8}\right) = 30$

C.3 Índice Maestro y Matriz Comparativa de los 35 Lenguajes de Programación (Parte V)

Problema resuelto [PRB-0013]

Un servidor de mensajería mantiene 10 000 conexiones concurrentes. Con hilos nativos del sistema operativo (pila reservada de 1 024 KiB por hilo), necesitaría 10 240 000 KiB. Con goroutines de Go o Virtual Threads de Java 21 (pila inicial dinámica de 4 KiB por tarea), calcule la memoria total de pila requerida en MiB (1 MiB = 1 024 KiB).

Respuesta

$$Resultado = 39{,}0625$$

Comprobación de la respuesta la aritmética del último paso reproduce la respuesta enunciada a la precisión con que está escrita. Es una reproducción aritmética; no valida el modelo físico ni sus hipótesis.

Ejercicio 1 [PRB-0015]

[CH-03.1] Calcule (Memoria en MiB requerida por 10 000 hilos nativos con pila de 1 MiB cada uno) expresado en MiB.

Respuesta 10000

Ejercicio 2 [PRB-0016]

[CH-03.2] Calcule (Factor de reducción de memoria al usar pilas iniciales de 4 KiB frente a 1024 KiB (1024 / 4)) expresado en veces.

Respuesta 256

Ejercicio 3 [PRB-0017]

[CH-03.3] Calcule (Número de lenguajes cubiertos en el Volumen VI (Java, Kotlin, Scala, C#, F#, Go, Swift, Objective-C)) expresado en lenguajes.

Respuesta 8

Ejercicio 4 [PRB-0018]

[CH-03.4] Calcule (Número de lenguajes cubiertos en el Volumen VII (SQL, Shell, PHP, Ruby, Dart + 6 funcionales + 6 especializados)) expresado en lenguajes.

Respuesta 17

Cálculos reproducidos

$\frac{(10000 \cdot 4)}{1024} = 39{,}0625$

C.4 Matriz Curricular (ACM/IEEE CS2023 y SWEBOK v4.0), Grafo de Prerrequisitos y Auditoría de Calidad

Problema resuelto [PRB-0019]
La colección completa de 10 volúmenes contiene 208 problemas en los Volúmenes I y II (104 cada uno), 48 problemas en el Volumen III (Rust), 200 problemas en los Volúmenes IV a VIII (40 cada uno), 35 problemas en el Volumen IX y 20 problemas en este Volumen 0. Calcule el número total de problemas verificados incluidos en la enciclopedia.
Respuesta

$$Resultado = 511$$

Comprobación de la respuesta la aritmética del último paso reproduce la respuesta enunciada a la precisión con que está escrita. Es una reproducción aritmética; no valida el modelo físico ni sus hipótesis.
Ejercicio 1 [PRB-0021]
[CH-04.1] Calcule (Total de capítulos entre los Volúmenes $I-$VIII (2x13 + 1x8 + 5x8 = 74) más Volumen IX (7) y Volumen 0 (4)) expresado en capítulos.
Respuesta 85
Ejercicio 2 [PRB-0022]
[CH-04.2] Calcule (Total de rutas de aprendizaje documentadas en blueprint/learning-paths.json) expresado en itinerarios.
Respuesta 19
Ejercicio 3 [PRB-0023]
[CH-04.3] Calcule (Proyectos integradores multifichero verificados en el directorio projects/) expresado en proyectos.
Respuesta 6
Ejercicio 4 [PRB-0024]
[CH-04.4] Calcule (Errores de auditoría de calidad (quality$_errors$) permitidos en cualquiera de los 10 volúmenes compilados) expresado en errores.
Respuesta 0

Cálculos reproducidos

$((((208 + 48) + 200) + 35) + 20) = 511$

Apéndice D

Auditoría de fuentes

Estado de las fuentes: PENDING. 27 pendientes. Los conflictos almacenados quedan fuera (0). Las fuentes que la guía almacenada no cita quedan fuera. Recuento de no citadas: 0.
¶1: dos testimonios.
¶2: dos testimonios.
¶3: dos testimonios.
¶4: dos testimonios.
¶5: dos testimonios.
¶6: dos testimonios.
¶7: dos testimonios.
¶8: dos testimonios. Ecuaciones: 1 valor o condición.
¶9: dos testimonios.
¶10: dos testimonios.
¶11: dos testimonios.
¶12: dos testimonios.
¶13: dos testimonios.
¶14: dos testimonios.
¶15: dos testimonios.
¶16: dos testimonios.
¶17: dos testimonios.
¶18: dos testimonios.
¶19: dos testimonios.
¶20: dos testimonios.
¶21: dos testimonios.
¶22: dos testimonios.
¶23: dos testimonios.
¶24: dos testimonios.

Computer Science Curricula 2023 (CS2023) & Guide to the Software Engineering Body of Knowledge (SWEBOK v4.0) (PENDING) — Suplemento abierto: — No es la bibliografía de la guía. — ¶1, ¶2, ¶3, ¶4, ¶5, ¶6, ¶7, ¶8, ¶9, ¶10, ¶11, ¶12, ¶13, ¶14, ¶15, ¶16, ¶17, ¶18, ¶19, ¶20, ¶21, ¶22, ¶23, ¶24.
The Art of Computer Programming & Structure and Interpretation of Computer Programs (PENDING) — Suplemento abierto: — No es la bibliografía de la guía. — ¶1, ¶2, ¶3, ¶4, ¶5, ¶6, ¶7, ¶8, ¶9, ¶10, ¶11, ¶12, ¶13, ¶14, ¶15, ¶16, ¶17, ¶18, ¶19, ¶20, ¶21, ¶22, ¶23, ¶24.
REV-0001: ch-01 scientific
REV-0002: ch-02 scientific
REV-0003: ch-03 scientific
REV-0004: ch-04 scientific

Apéndice E

Plan de estudio

Volumen Maestro Integral (Partes I a XIII)

✓Arquitectura General de la Enciclopedia y Mapa de las 13 Partes (Volúmenes I a IX)

- Comprender la estructura modular de las 13 Partes y los 10 volúmenes de la enciclopedia, seleccionar el volumen o itinerario adecuado y utilizar las convenciones tipográficas y de verificación.

2 conceptos planificados.

✓Síntesis de las Partes I a IV: Historia, Fundamentos Universales, Algoritmos y Sistemas

- Conectar la evolución histórica de los paradigmas de programación con la representación física de la información, el análisis de algoritmos y la arquitectura de computadores y compiladores.

2 conceptos planificados.

✓Índice Maestro y Matriz Comparativa de los 35 Lenguajes de Programación (Parte V)

- Comparar los 35 lenguajes de programación de la enciclopedia según su sistema de tipos, modelo de memoria, concurrencia, modelo de ejecución y dominios óptimos de aplicación.

2 conceptos planificados.

✓Matriz Curricular (ACM/IEEE CS2023 y SWEBOK v4.0), Grafo de Prerrequisitos y Auditoría de Calidad

- Verificar la correspondencia de cada volumen con las áreas de conocimiento de CS2023 y SWEBOK v4.0, navegar el DAG de prerrequisitos y auditar los artefactos reproducibles del proyecto.

2 conceptos planificados.

Bibliografía

[1] Alan M. Turing – On Computable Numbers, with an Application to the Entscheidungsproblem (1936). https://www.cs.virginia.edu/~robins/Turing_Paper_1936.pdf

[2] John von Neumann – First Draft of a Report on the EDVAC (1945). https://web.mit.edu/STS.035/www/PDFs/edvac.pdf

[3] IEEE Standard for Floating-Point Arithmetic (IEEE 754-2019) / Wikipedia Summary. https://en.wikipedia.org/wiki/IEEE_754

[4] RFC 3629 – UTF-8, a transformation format of ISO 10646 (IETF Standard). https://datatracker.ietf.org/doc/html/rfc3629

[5] ACM / IEEE-CS / AAAI Computer Science Curricula 2023 (CS2023). https://csed.acm.org/

[6] MIT OpenCourseWare 6.006 – Introduction to Algorithms. https://ocw.mit.edu/courses/6-006-introduction-to-algorithms-spring-2020/

[7] The Rust Programming Language (Klabnik & Nichols – Official Rust Documentation). https://doc.rust-lang.org/book/

[8] The Rust Reference – Memory Model, Type System, Lifetimes, and Unsafe Rust. https://doc.rust-lang.org/reference/

[9] The Rustonomicon – The Dark Arts of Unsafe Rust. https://doc.rust-lang.org/nomicon/

[10] C Programming Language Reference (ISO/IEC 9899 C17/C23 – cppreference). https://en.cppreference.com/w/c

[11] C++ Reference (ISO/IEC 14882 C++20/C++23 – cppreference). https://en.cppreference.com/w/cpp

[12] The Python Language Reference & Standard Library Documentation (Python 3.12–3.14). https://docs.python.org/3/reference/index.html

[13] ECMA-262 – ECMAScript 2024 Language Specification. https://tc39.es/ecma262/

[14] The TypeScript Handbook – Structural Typing, Narrowing, and Type Manipulation. https://www.typescriptlang.org/docs/handbook/intro.html

[15] Microsoft .NET & C# Language Specification (ECMA-334 / .NET 8 & 9). https://learn.microsoft.com/en-us/dotnet/csharp/

[16] The Java Language Specification & JVM Specification (Java SE 21 LTS). https://docs.oracle.com/javase/specs/jls/se21/html/index.html

[17] The Go Programming Language Specification & Memory Model. https://go.dev/ref/spec

[18] Kotlin Language Documentation & Coroutines Guide (Kotlin 2.0). https://kotlinlang.org/docs/home.html

[19] The Swift Programming Language (Swift 6.0 Official Book). https://docs.swift.org/swift-book/documentation/the-swift-programming-language/

[20] E. F. Codd – A Relational Model of Data for Large Shared Data Banks (1970) & PostgreSQL Docs. https://www.postgresql.org/docs/current/

[21] SQLite 3.54 Transactional SQL Database Engine Documentation. https://www.sqlite.org/docs.html

[22] GNU Bash Reference Manual & IEEE Std 1003.1 POSIX Shell Specification. https://www.gnu.org/software/bash/manual/bash.html

[23] WHATWG HTML Living Standard & W3C CSS Specifications. https://html.spec.whatwg.org/multipage/

[24] RFC 8259 – The JavaScript Object Notation (JSON) Data Interchange Format & YAML 1.2.2. https://datatracker.ietf.org/doc/html/rfc8259

[25] IEEE Computer Society – Guide to the Software Engineering Body of Knowledge (SWEBOK v4.0). https://www.computer.org/education/bodies-of-knowledge/software-engineering

[26] Computer Science Curricula 2023 (CS2023) & Guide to the Software Engineering Body of Knowledge (SWEBOK v4.0). https://csed.acm.org/vol-ref-1

[27] The Art of Computer Programming & Structure and Interpretation of Computer Programs. https://mitp-content-server.mit.edu/books/content/sectbyfn/books_pres_0/6515/sicp.zip/index.html/vol-ref-2